

Finite difference sensitivity verification in gradient-based optimization

Mattias Schevenels

January 2026

1 Introduction

Gradient-based optimization algorithms require accurate sensitivities of the objective and constraint functions with respect to the design variables. These sensitivities are typically derived analytically, for instance using the direct method or the adjoint method, and subsequently implemented in computer code. Errors may occur both in the analytical derivation of the required expressions and in their numerical implementation.

Incorrect sensitivities may lead to slow convergence, convergence to incorrect solutions, or complete failure of the optimization process. Sensitivity verification therefore forms an essential part of the development and validation of optimization codes. In this technical note, three finite difference strategies for sensitivity verification are discussed: (1) component-wise finite difference verification, (2) directional finite difference verification, and (3) Taylor finite difference verification. In all three approaches, analytically computed sensitivities are verified against numerical finite difference approximations, which are used as reference values.

Throughout this note, we consider a design vector $\mathbf{x} \in \mathbb{R}^n$, an objective function $f_0(\mathbf{x})$, and constraint functions $f_i(\mathbf{x})$, $i = 1, \dots, m$. The sensitivities $\partial f_i / \partial \mathbf{x}$ of the objective and the constraints are verified for one specific value of the design vector, denoted as $\mathbf{x}_{\text{base}}$.

2 Component-wise finite difference verification

Principle

In a component-wise finite difference verification, all components $\partial f_i / \partial x_j$ of the sensitivities of the objective $f_0(\mathbf{x})$ and the constraints $f_i(\mathbf{x})$ are compared with their finite difference approximations. Commonly, forward finite differences are used, which implies that the objective function and the constraints must be evaluated for n additional design vectors given by $\mathbf{x}_{\text{base}} + h \mathbf{e}_j$, where h is a small perturbation step and $\mathbf{e}_j$ denotes the j -th unit vector.

The verification is then performed as follows:

$$\frac{\partial f_i}{\partial x_j} \approx \frac{f_i(\mathbf{x} + h\mathbf{e}_j) - f_i(\mathbf{x})}{h} \quad (1)$$

A suitable choice of the perturbation step h is essential. If h is too large, the finite difference approximation becomes inaccurate due to truncation errors. If h is too small (relative to machine precision), numerical round-off errors may dominate and lead to noisy results. It is therefore recommended to perform the verification for a range of step sizes spanning several orders of magnitude.

MATLAB implementation

This subsection illustrates how a component-wise finite difference verification can be implemented in MATLAB, in combination with MMA/GCMMA (implementations in which the user explicitly controls the optimization loop).

Listing 1: Component-wise finite difference verification code, initialization block.

```
% COMPONENT-WISE FINITE DIFFERENCE VERIFICATION PARAMETERS
iterFDcheck = 1; % Start verification at this iteration
FDvar = [1,2,6,7]; % Design variables to check
nFDvar = numel(FDvar); % Number of design variables to check
h = logspace(-2,-16,15); % Step sizes for perturbations dx = h*e_j
nFDstep = numel(h); % Number of step sizes
df0dx_fd = zeros(1,nFDvar,nFDstep); % (f0(x+dx)-f0(x))/h
dfdx_fd = zeros(m,nFDvar,nFDstep); % (f(x+dx)-f(x))/h
```

The code shown in listing 1 must be placed before the optimization loop. The variable `iterFDcheck` determines the iteration of the optimization loop at which the optimization is interrupted and the sensitivity verification is started. Setting `iterFDcheck` to 1 performs the verification for the initial design. Setting it to `inf` disables the sensitivity verification.

The row vector `FDvar` collects the indices of the design variables for which the verification is performed. For problems with many design variables, performing a component-wise finite difference verification for all variables may be computationally expensive. In such cases, it is often sufficient to verify a representative subset of variables. This is particularly justified when all variables are treated in an identical manner in the code, for example through the use of vectorized operations. In that case, it is unlikely that the sensitivities for some variables are implemented correctly while others are not. For problems involving multiple types of design variables (e.g. size and shape variables), it is advisable to include one or more variables of each type in the verification.

The variable `h` defines the perturbation step size. If it is difficult to estimate an appropriate order of magnitude for this parameter, multiple values may be used; in that case, `h` should be defined as a row vector, as shown in the example.

The arrays `df0dx_fd` and `dfdx_fd`, preallocated here, will store the finite difference approximations of the sensitivities of the objective function and the constraints, respectively. The variable `m` denotes the number of constraints and is assumed to be defined earlier.

The code in listing 2 must be placed inside the optimization loop, before the call to MMA/GCMMA. This code block assumes that the following variables have been defined earlier: the iteration counter `iter`, the current design `x`, the corresponding values of the objective function `f0` and constraints `f`, as well as their

Listing 2: Component-wise finite difference verification code, loop block.

```
% COMPONENT-WISE FINITE DIFFERENCE VERIFICATION
if iter>=iterFDcheck
    iFDtest = iter-iterFDcheck;
    if iFDtest==0
        x_base = x;
        f0_base = f0;
        f_base = f;
        df0dx_base = df0dx;
        dfdx_base = dfdx;
    else
        iFDvar = floor((iFDtest-1)/nFDstep)+1;
        iFDstep = mod(iFDtest-1,nFDstep)+1;
        df0dx_fd(1,iFDvar,iFDstep) = (f0-f0_base)/h(iFDstep);
        dfdx_fd(:,iFDvar,iFDstep) = (f-f_base)/h(iFDstep);
    end
    if iFDtest<nFDvar*nFDstep
        iFDvar = floor(iFDtest/nFDstep)+1;
        iFDstep = mod(iFDtest,nFDstep)+1;
        xnew = x_base;
        xnew(FDvar(iFDvar)) = x_base(FDvar(iFDvar)) + h(iFDstep);
        if any(xnew<xmin) || any(xnew>xmax)
            error('FD check invalid: x_base + h(%i)*e_%i violates box constraints.', ...
                iFDstep,FDvar(iFDvar));
        end
        x = xnew;
        iter = iter+1;
        continue;
    else
        printcomponentfdcheck(h,df0dx_base(FDvar),dfdx_base(:,FDvar),df0dx_fd,dfdx_fd,FDvar);
        break;
    end
end
```

Note: when copying code from the PDF, some PDF viewers may remove leading indentation; if this happens, paste the code into the MATLAB Editor and use Ctrl+I (Smart Indent) to re-indent it.

sensitivities $df0dx$ and $dfdx$, and the box constraints $xmin$ and $xmax$. The code stores the design from iteration $iterFDcheck$ as the base design. The objective function, the constraints, and the analytical sensitivities at this design point are also stored; these analytical sensitivities (for the base design) are the quantities that will be verified using finite differences. In subsequent iterations, the design is perturbed along the selected design variables and for all specified step sizes. The objective function and the constraints are recomputed (elsewhere in the loop) and used to calculate the finite difference approximations. At the end of this process, the function `printcomponentfdcheck` from the `StructOpt` toolbox is called to print the verification results to the MATLAB command window.

Example output

The function `printcomponentfdcheck` in the `StructOpt` toolbox prints the results of the component-wise finite difference verification to the MATLAB command window. Example output is shown in listing 3. The first column lists the step sizes used for the finite difference approximations. The remaining columns show the analytical sensitivities, the corresponding finite difference approximations, and the absolute differences between both. For this example, the finite difference approximations are stable for step sizes h ranging approximately from 10^{-5} to 10^{-11} . In this range, the finite difference results can be considered reliable reference values. The close agreement between these numerical reference values and the analytical sensitivities indicates that the analytical sensitivities are computed correctly.

Listing 3: Component-wise finite difference verification results.

h	df0dx1	df0dx1_fd	diff	df0dx2	df0dx2_fd	diff
1.0000e-02	1.7071e+01	1.7139e+01	6.7588e-02	1.7071e+01	1.7189e+01	1.1759e-01
1.0000e-03	1.7071e+01	1.7078e+01	6.7669e-03	1.7071e+01	1.7083e+01	1.1767e-02
1.0000e-04	1.7071e+01	1.7072e+01	6.7677e-04	1.7071e+01	1.7072e+01	1.1768e-03
1.0000e-05	1.7071e+01	1.7071e+01	6.7679e-05	1.7071e+01	1.7071e+01	1.1768e-04
1.0000e-06	1.7071e+01	1.7071e+01	6.7687e-06	1.7071e+01	1.7071e+01	1.1771e-05
1.0000e-07	1.7071e+01	1.7071e+01	8.0017e-07	1.7071e+01	1.7071e+01	1.0844e-06
1.0000e-08	1.7071e+01	1.7071e+01	2.7897e-06	1.7071e+01	1.7071e+01	2.7897e-06
1.0000e-09	1.7071e+01	1.7071e+01	5.6319e-06	1.7071e+01	1.7071e+01	5.6319e-06
1.0000e-10	1.7071e+01	1.7071e+01	4.3196e-04	1.7071e+01	1.7071e+01	4.3196e-04
1.0000e-11	1.7071e+01	1.7070e+01	9.8913e-04	1.7071e+01	1.7070e+01	9.8913e-04
1.0000e-12	1.7071e+01	1.7081e+01	1.0380e-02	1.7071e+01	1.7081e+01	1.0380e-02
1.0000e-13	1.7071e+01	1.7337e+01	2.6617e-01	1.7071e+01	1.7337e+01	2.6617e-01
1.0000e-14	1.7071e+01	1.7053e+01	1.8042e-02	1.7071e+01	1.7053e+01	1.8042e-02
1.0000e-15	1.7071e+01	0.0000e+00	1.7071e+01	1.7071e+01	0.0000e+00	1.7071e+01
1.0000e-16	1.7071e+01	0.0000e+00	1.7071e+01	1.7071e+01	0.0000e+00	1.7071e+01

3 Directional finite difference verification

Principle

In a directional finite difference verification, the sensitivities of the objective and constraint functions are verified along a single perturbation direction $\mathbf{p} \in \mathbb{R}^n$. Rather than perturbing individual design variables, the design vector is perturbed as $\mathbf{x}_{\text{base}} + h\mathbf{p}$, where h is a scalar step size and $\mathbf{p}$ is a prescribed direction vector, typically chosen at random and normalized in an appropriate manner. The analytical directional derivative is then compared with the finite difference approximation:

$$\left(\frac{\partial f_i}{\partial \mathbf{x}}\right) \mathbf{p} \stackrel{?}{=} \frac{f_i(\mathbf{x}_{\text{base}} + h\mathbf{p}) - f_i(\mathbf{x}_{\text{base}})}{h}. \quad (2)$$

Directional finite difference verification is well established in the literature on nonlinear optimization and is discussed, for instance, by Gill, Murray, and Wright [2]. It is particularly attractive for problems with a large number of design variables, since its computational cost is essentially independent of the problem dimension.

MATLAB implementation

Listing 4: Directional finite difference verification code, initialization block.

```
% FINITE DIFFERENCE VERIFICATION PARAMETERS
iterFDcheck = 1; % Start verification at this iteration
FDvar = [1,2,6,7]; % Design variables to check
nFDvar = numel(FDvar); % Number of design variables to check
h = logspace(-2,-16,15); % Step sizes for perturbations dx = h*e_j
nFDstep = numel(h); % Number of step sizes
df0dx_fd = zeros(1,nFDvar,nFDstep); % (f0(x+dx)-f0(x))/h
dfdx_fd = zeros(m,nFDvar,nFDstep); % (f(x+dx)-f(x))/h
```

The directional finite difference verification is implemented in a similar way as the component-wise verification. The code in listing 4 must be placed before the optimization loop. This code block assumes that the following variables have been defined earlier: the number of design variables n , the number of constraints m , and the

lower and upper box constraints $x_{\min}$ and $x_{\max}$. The variable p defines the perturbation direction. In this case, each component is normalized using the distance between the box constraints. Normalization ensures that the magnitude of the perturbation is controlled exclusively by the step size h . As in the component-wise case, it is generally advisable to consider a range of step sizes spanning several orders of magnitude. The arrays `df0dxdp_fd` and `dfdxp_fd`, preallocated here, will store the finite difference approximations of the directional derivatives of the objective function and the constraints, respectively.

Listing 5: Directional finite difference verification code, loop block.

```
% DIRECTIONAL FINITE DIFFERENCE VERIFICATION
if iter>=iterFDcheck
    iFDstep = iter-iterFDcheck;
    if iFDstep==0
        x_base = x;
        f0_base = f0;
        f_base = f;
        df0dxdp_base = df0dx*p;
        dfdxp_base = dfdx*p;
    else
        df0dxdp_fd(iFDstep) = (f0-f0_base)/h(iFDstep);
        dfdxp_fd(:,iFDstep) = (f-f_base)/h(iFDstep);
    end
    if iFDstep<nFDstep
        dx = h(iFDstep+1)*p;
        xnew = x_base+dx;
        if any(xnew<xmin) || any(xnew>xmax)
            error('FD check invalid: x_base + h(%i)*p violates box constraints.',iFDstep+1);
        end
        x = xnew;
        iter = iter+1;
        continue;
    else
        printdirectionalfdcheck(h,df0dxdp_base,dfdxp_base,df0dxdp_fd,dfdxp_fd)
        break;
    end
end
```

The code in listing 5 must be placed inside the optimization loop, before the MMA/GCMMMA call. This code block assumes that the following variables have been defined earlier: the iteration counter `iter`, the current design `x`, the corresponding values of the objective function `f0` and constraints `f`, and their sensitivities `df0dx` and `dfdx`. The code stores the design from iteration `iterFDcheck` as the base design. The objective function and the constraints this design point are also stored, and the analytical directional derivatives `df0dxdp_base` and `dfdxp_base` along p are calculated. In subsequent iterations, the design vector is perturbed along p using the specified step sizes. The resulting function values are used to compute the finite difference approximations `df0dxdp_fd` and `dfdxp_fd` of the directional derivatives. At the end of the process, the function `printdirectionalfdcheck` from the `StructOpt` toolbox is called to print the verification results.

Example output

The function `printdirectionalfdcheck` prints the analytical directional derivatives, the corresponding finite difference approximations, and the absolute differences between both. Example output is shown in listing 6. Agreement between the analytical directional derivatives and the numerical finite difference values in the range of step sizes where the latter stabilize – here between 10^{-6} and 10^{-12} – indicates that the analytical sensitivities are implemented correctly.

Listing 6: Directional finite difference verification results.

h	df0dxdp	df0dxdp_fd	diff	df1dxdp	df1dxdp_fd	diff
1.0000e-02	-4.6763e+02	-3.6651e+02	1.0112e+02	3.3633e+02	1.0217e+03	6.8537e+02
1.0000e-03	-4.6763e+02	-4.5608e+02	1.1550e+01	3.3633e+02	3.8945e+02	5.3120e+01
1.0000e-04	-4.6763e+02	-4.6646e+02	1.1681e+00	3.3633e+02	3.4153e+02	5.2071e+00
1.0000e-05	-4.6763e+02	-4.6751e+02	1.1694e-01	3.3633e+02	3.3685e+02	5.1970e-01
1.0000e-06	-4.6763e+02	-4.6761e+02	1.1695e-02	3.3633e+02	3.3638e+02	5.1961e-02
1.0000e-07	-4.6763e+02	-4.6763e+02	1.1696e-03	3.3633e+02	3.3633e+02	5.2103e-03
1.0000e-08	-4.6763e+02	-4.6763e+02	1.1714e-04	3.3633e+02	3.3633e+02	5.3831e-04
1.0000e-09	-4.6763e+02	-4.6763e+02	6.0621e-07	3.3633e+02	3.3633e+02	4.1275e-04
1.0000e-10	-4.6763e+02	-4.6763e+02	4.2572e-04	3.3633e+02	3.3633e+02	3.4988e-03
1.0000e-11	-4.6763e+02	-4.6763e+02	4.4048e-03	3.3633e+02	3.3655e+02	2.2576e-01
1.0000e-12	-4.6763e+02	-4.6768e+02	5.2722e-02	3.3633e+02	3.3671e+02	3.8492e-01
1.0000e-13	-4.6763e+02	-4.6782e+02	1.9483e-01	3.3633e+02	3.2983e+02	6.4931e+00
1.0000e-14	-4.6763e+02	-4.6896e+02	1.3317e+00	3.3633e+02	2.9914e+02	3.7189e+01
1.0000e-15	-4.6763e+02	-5.1159e+02	4.3964e+01	3.3633e+02	1.5490e+03	1.2127e+03
1.0000e-16	-4.6763e+02	-8.5265e+02	3.8502e+02	3.3633e+02	-4.9383e+03	5.2746e+03

4 Taylor finite difference verification

Principle

Taylor finite difference verification, or the Taylor remainder convergence test, is closely related to the directional finite difference verification discussed in the previous section. Rather than directly comparing directional derivatives along $\mathbf{p}$, this approach is based on a first-order Taylor expansion of the objective and constraint functions around a reference design $\mathbf{x}_{\text{base}}$ in the direction of $\mathbf{p}$.

For a perturbation of the form $\mathbf{x} = \mathbf{x}_{\text{base}} + h \mathbf{p}$, the first-order Taylor expansion of $f_i(\mathbf{x})$ reads:

$$f_i(\mathbf{x}_{\text{base}} + h \mathbf{p}) = f_i(\mathbf{x}_{\text{base}}) + h \left(\frac{\partial f_i}{\partial \mathbf{x}} \right) \mathbf{p} + \mathcal{O}(h^2) \quad (3)$$

Rearranging terms leads to the Taylor remainder convergence test:

$$f_i(\mathbf{x}_{\text{base}} + h \mathbf{p}) - f_i(\mathbf{x}_{\text{base}}) - h \left(\frac{\partial f_i}{\partial \mathbf{x}} \right) \mathbf{p} \stackrel{?}{=} \mathcal{O}(h^2) \quad (4)$$

The residual of the first-order Taylor series expansion, on the left-hand side of this expression, is referred to as the second-order Taylor remainder. The Taylor test examines whether this remainder effectively decreases quadratically with the step size h . Note that the second-order Taylor remainder involves exactly the same quantities as those used in the directional finite difference verification, multiplied by the step size h . Whereas the directional finite difference verification simply checks whether this remainder tends to zero for small values of h , the Taylor test additionally assesses the rate at which this convergence occurs. Taylor finite difference verification therefore provides a more stringent verification of the analytically computed sensitivities. It is widely used for this purpose in the literature; see, for example, Farrell et al. [1].

MATLAB implementation

The implementation of the Taylor finite difference verification closely follows the structure of the directional finite difference verification discussed in the previous section. The code in listing 7 must be placed before the optimization loop. This code block assumes that the following variables have been defined earlier: the number of design variables n , the number of constraints m , and the lower and upper box constraints $\mathbf{x}_{\text{min}}$ and $\mathbf{x}_{\text{max}}$. As

Listing 7: Taylor finite difference verification code, initialization block.

```
% TAYLOR FINITE DIFFERENCE VERIFICATION PARAMETERS
iterFDcheck = 1; % Start verification at this iteration
p = randn(n,1).*(xmax-xmin); % Perturbation direction
h = logspace(-2,-12,50); % Step sizes for perturbations dx = h*p
nFDstep = numel(h); % Number of step sizes
r0 = zeros(1,nFDstep); % |f0(x+dx)-f0(x)-df0dx*dx|
r = zeros(m,nFDstep); % |f(x+dx)-f(x)-dfdx*dx
```

in the directional verification, a fixed perturbation direction p is selected and scaled using the box constraints in order to ensure that all design variables contribute in a balanced manner. The step sizes h are sampled over several orders of magnitude; in this case a finer sampling is used to allow for a clearer assessment of the convergence rate in the subsequent log–log plot. The arrays $r0$ and r are preallocated to store the absolute values of the Taylor remainders for the objective function and the constraints, respectively.

Listing 8: Taylor finite difference verification code, loop block.

```
% TAYLOR FINITE DIFFERENCE VERIFICATION
if iter>=iterFDcheck
    iFDstep = iter-iterFDcheck;
    if iFDstep==0
        x_base = x;
        f0_base = f0;
        f_base = f;
        df0dx_base = df0dx;
        dfdx_base = dfdx;
    else
        r0(iFDstep) = abs(f0-f0_base-df0dx*dx);
        r(:,iFDstep) = abs(f-f_base-dfdx*dx);
    end
    if iFDstep<nFDstep
        dx = h(iFDstep+1)*p;
        xnew = x_base+dx;
        if any(xnew<xmin) || any(xnew>xmax)
            error('FD check invalid: x_base + h(%i)*p violates box constraints.',iFDstep+1);
        end
        x = xnew;
        iter = iter+1;
        continue;
    else
        figure;
        plottaylorfdcheck(h,r0,r)
        break;
    end
end
end
```

The code in listing 8 must be placed inside the optimization loop, before the call to MMA/GCMMA. This code block assumes that the following variables have been defined earlier: the iteration counter $iter$, the current design x , the corresponding values of the objective function $f0$ and constraints f , and their sensitivities $df0dx$ and $dfdx$. At the iteration specified by $iterFDcheck$, the current design is stored as the base design, together with the corresponding function values and analytical sensitivities.

In subsequent iterations, the design vector is perturbed along the fixed direction p using the prescribed step sizes. For each perturbation, the objective and constraint functions are re-evaluated, and the Taylor remainders are computed by subtracting the first-order Taylor approximation from the exact function values. These remainders are stored in the arrays $r0$ and r .

Once all perturbation steps have been processed, the function `plottaylorfdcheck` from the `StructOpt` toolbox is called. This function generates a log–log plot of the Taylor remainders as a function of the step size h , together with a reference curve proportional to h^2 . The resulting plot provides a clear visual assessment of the expected second-order convergence behaviour.

Example result

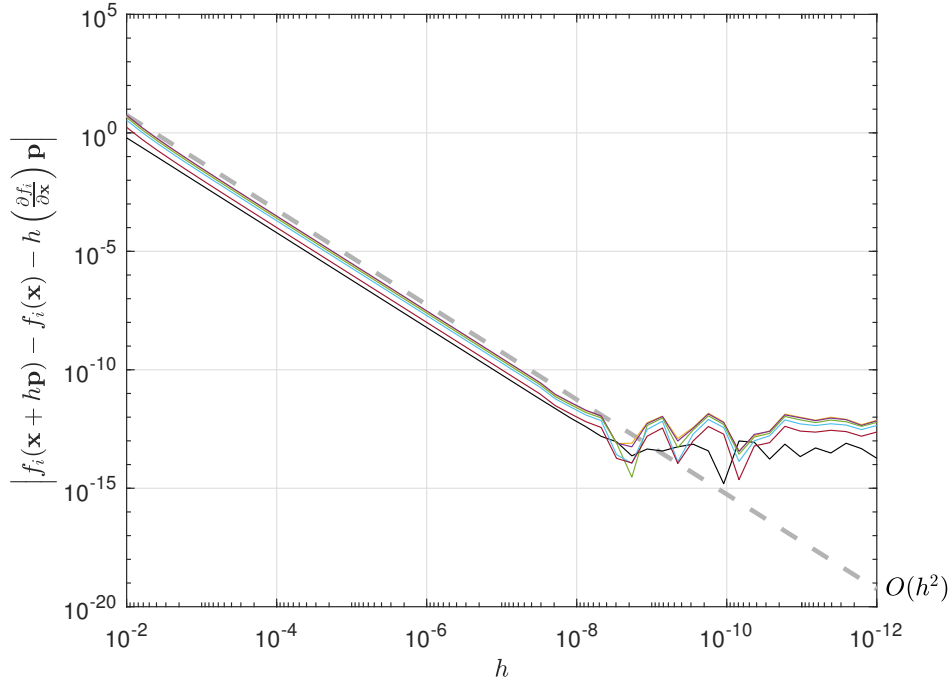


Figure 1: Second-order Taylor remainder for the objective (black curve) and the constraints (colored curves). The reference $O(h^2)$ curve is shown for comparison.

Figure 1 shows a typical result of a Taylor finite difference verification. For an intermediate range of step sizes h , the Taylor remainder decreases proportionally to h^2 , as predicted by the first-order Taylor expansion. This behavior indicates that the analytically computed sensitivities are consistent with the finite difference perturbations and are therefore implemented correctly.

For larger values of h , small deviations from the quadratic convergence rate are observed due to the increasing influence of higher-order terms in the Taylor expansion. Conversely, for very small values of h , numerical round-off errors dominate, causing the Taylor remainder to deviate strongly from the ideal $O(h^2)$ behaviour. The presence of a clear intermediate range with quadratic convergence is the key indicator of a successful Taylor finite difference verification.

It should be noted that the characteristic $O(h^2)$ convergence behaviour does not appear when the objective or constraint function is exactly linear in the design variables. In that case, the first-order Taylor expansion is exact and the Taylor remainder is identically zero for all step sizes h , apart from small numerical round-off errors.

5 Conclusion

Sensitivity verification is an essential step in the development and validation of gradient-based optimization codes. In this technical note, three finite difference strategies are discussed: component-wise, directional, and Taylor finite difference verification. Each method probes the consistency between analytically computed sensitivities and numerical function evaluations from a different perspective.

Component-wise finite difference verification provides the most direct and transparent check, as it compares individual sensitivity components with their finite difference approximations along selected coordinate directions. This approach is particularly appropriate when different types of design variables are present, or when sensitivities are derived or implemented in different ways for different variables. In such cases, component-wise verification helps to isolate errors at the level of individual design variables.

Directional and Taylor finite difference verifications perturb the design vector along one or more prescribed directions in the design space. When all design variables are treated in an identical manner in the analytical derivation and in the numerical implementation of the sensitivities – for instance through vectorized operations – these approaches provide an efficient and reliable alternative to component-wise verification. Their computational cost is essentially independent of the number of design variables.

Taylor finite difference verification can be regarded as a stricter variant of the directional approach, as it examines not only consistency but also the expected second-order convergence of the Taylor remainder. At the same time, it is somewhat less intuitive to interpret than direct comparisons of analytical and finite difference sensitivities.

References

- [1] P.E. Farrell, D.A. Ham, S.W. Funke, and M.E. Rognes. Automated derivation of the adjoint of high-level transient finite element programs. *SIAM Journal on Scientific Computing*, 35(4):C369–C393, 2013.
- [2] P.E. Gill, W. Murray, and M.H. Wright. *Practical Optimization*. Academic Press, London; New York, 1981.